ETC3250/5250 Introduction to Machine Learning

Week 6: Neural networks and deep learning

Jack Jewson

Jack.Jewson@monash.edu

Department of Econometrics and Business Statistics

ETC3250/5250 Lecture 6

Overview

We will cover:

- Structure of a neural network
- Fitting neural networks
- Diagnosing the fit

ETC3250/5250 Lecture 6

Structure of a neural network

ETC3250/5250 Lecture 6

Linear regression as a network

Consider a linear regression model regressing a continuous response $y \in \mathbb{R}$, on predictors $x \in \mathbb{R}^p$

$$\hat{y} = \mathbb{E}[y \mid x] = \beta_0 + \sum_{j=1}^{p} \beta_j x_j$$

We can draw this as a network model:

- p inputs (predictors),
- multiplied by weights (coefficients),
- summed, add a constant,
- predicts output (response).

ETC3250/5250 Lecture 6

Difference between linear and logistic regression

Logistic regression, predicting the probability of a binary outcome ,

$$P(Y = 1) = \frac{1}{1 + e^{-(\beta_0 + \sum_{j=1}^{p} \beta_j x_j)}}$$

Can also be written as a network model

- p inputs (predictors),
- multiplied by weights (coefficients),
- summed, add a constant,
- Applied a function to this in order to model a probability
- predicts output (response).

ETC3250/5250 Lecture 6

Neural Network Notation

Both linear and logistic regression are **simple** one layered neural networks

- Inputs Layer where the predictors are fed in,
- Output Layer where the predicted value from the model is fed out
- The connection between any node j in the input layer and node k in the output layer is weighted by a parameter b_{jk},
- Nodes which take the weighted sum of their inputs $\sum_{j=1}^{p} b_{jk}x_j$ add a bias b_{0k} and return a function applied to this

---

ETC3250/5250 Lecture 6

Neural Network Notation

Both linear and logistic regression are **simple** neural networks

- These NNs only have one node and its an output node, so it uses an output function $\sigma(\cdot)$.
  - In Linear regression $\sigma(a) = a$
  - In Logistic regression $\sigma(a) = \frac{1}{1+e^{-a}}$

ETC3250/5250 Lecture 6

Hidden layers

So neural networks are just simple linear models? **Absolutely not**

- Introduce a Hidden Layer between the inputs (predictors) and outputs (the machine learning model prediction),
- Each node in the hidden layer takes the weighted sum of the inputs $\sum_{j=1}^{p} b_{jk} x_j$ add a bias b_{0k} and return a function applied to this
- For hidden layers this function is called an activation function $\varphi(\cdot)$.
  - This must be a non-linear function (as every other part of the NN is linear)

ETC3250/5250 Lecture 6

Single hidden layer NN

Consider s hidden nodes in a single hidden layer

- Every input node is connected to every hidden node - with weight b_{jk} $j = 1, \ldots p$, $k = 1, \ldots, s$
- Every hidden node is connected to every output node with weight a_{jk} $j = 1, \ldots s$ $k = 1$
- The output of each hidden node is $h_k = \varphi(b_{0k} + \sum_{j=1}^{p} b_{jk} x_j)$
- The machine learning model prediction is $\sigma(a_{01} + \sum_{j=1}^{s} a_{j1} h_j)$

ETC3250/5250 Lecture 6

Single hidden layer NN

Consider s hidden nodes in a single hidden layer

$$g(y; x) = \sigma(a_{01} + \sum_{k=1}^{s} a_{k1} \varphi(b_{0k} + \sum_{j=1}^{p} b_{jk} x_j))$$

This is referred to as a fully connected feed forward neural network with 1 hidden layer (sometimes referred to as having 2 layers)

ETC3250/5250 Lecture 6

Feature Engineering

The hidden layers are learning features, i.e. combinations of the predictor variables, that best predict the response

Logistic regression

$$P(Y = 1 \mid x) = \frac{1}{1 + e^{-x^\top \beta}}$$

- $x \in \mathbb{R}^p$ are predictor variables
- $\beta \in \mathbb{R}^p$ regression coefficients

Single Hidden Layer NN

$$P(Y = 1 \mid x) = \frac{1}{1 + e^{-h^\top a}}$$

- $h \in \mathbb{R}^s$ are hidden layer outputs $h_k = \varphi(b_{0k} + \sum_{j=1}^{p} b_{jk} x_j)$
- with parameters b
- $a \in \mathbb{R}^p$ coefficients that connect the encoing of the predictors with the response

ETC3250/5250 Lecture 6

- Can view h as your transformed features that were learned to best predict y from x

ETC3250/5250 Lecture 6

Activation functions

- The output function $\sigma(\cdot)$ is given by the problem
- The activation function is part of the model's architecture
- The activation function **MUST** be non-linear - a NN with linear activation functions would just be an overparametrised linear model
- Popular examples are
  - $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
  - $\mathrm{ReLU}(x) = \max(0, x)$

ETC3250/5250 Lecture 6

What does this look like? (1/2)

The architecture allows for combining multiple linear models to generate non-linear classifications.

The best fit uses $s = 4$, four nodes in the hidden layer. Can you sketch four lines that would split this data well?

Wickham et al (2015) Removing the Blindfold

ETC3250/5250 Lecture 6

What does this look like? (2/2)

The models at each of the nodes of the hidden layer. Plotting $\mathrm{h_j}$.

The parameters a of the connection between the hidden layer and the output layer learn how to combine these 4 '*models*' into one prediction

ETC3250/5250 Lecture 6

Model fitting

The Neural Network architecture defines the number of hidden layers, the number of nodes in each layer and the activation functions. These are the NN hyperparameters.

For fixed architecture (#Hidden Layers = 1, #Nodes = s), fitting the model requires learning the best values for parameters $b_0 \in \mathbb{R}^s$, $B \in \mathbb{R}^{p,s}$ (with elements b_{jk}), $a_0 \in \mathbb{R}$, and $a \in \mathbb{R}^s$

These are all just numbers! We learn them exactly how we learned the regression parameters in linear models, using Maximum Likelihood

ETC3250/5250 Lecture 6

Model fitting

Linear Models

- Linear Regression:
  - $E[Y \mid x] = \beta_0 + \sum_{j=1}^{p} x_j \beta_j$
  - β's learned via least squares
- Logistic Regression:
  - $P(Y = 1 \mid x) = \frac{1}{1+e^{-\beta_0 - \sum_{j=1}^{p} x_j \beta_j}}$
  - β's learned via MLE

ETC3250/5250 Lecture 6

Model fitting

Single Layer Neural Network

- Regression:
  - $E[Y \mid x] = a_{01} + \sum_{k=1}^{s} a_{k1}\varphi(b_{0k} + \sum_{j=1}^{p} b_{jk}x_j)$
  - Learn a's and b's to minimise the squared error
- Binary Classification:
  - $P(Y = 1 \mid x) = \frac{1}{1+e^{-a_{01}-\sum_{k=1}^{s} a_{k1}\varphi(b_{0k}+\sum_{j=1}^{p} b_{jk}x_j)}}$
  - Learn a's and b's that make the observed data the most likely

ETC3250/5250 Lecture 6

Model fitting

There is one key difference in how linear models and neural networks are fit.

In linear models the exact MLE is found. If you fit the model twice you get exactly the same fit.

In Neural Network models the MLE is not found exactly. If you fit the model twice you will get a different fit.
This is a result of NN's having many more parameters and as a resault being much more complicated prediction functions

- The linear models have $p + 1$ parameters.
- The Single Hidden Layer NN has $(p + 1) \times s + (s + 1)$

ETC3250/5250 Lecture 6

But can be painful to find the best!

The same models with $s = 2, 3, 4$ fit several times.

Fitted using the R package `nnet`. It's very unstable, and this is still a problem with current procedures.

Fitting with keras

ETC3250/5250 Lecture 6

Steps (1/2)

1. Define architecture

- How many hidden layers do you need?
- How many nodes in the hidden layer?

2. Specify activation function: relu (rectified linear unit) or tanh, and output function linear, sigmoid, softmax

3. Choose loss function:

- MSE: squared difference between observation and prediction (also the log-likleihood under the Gaussian distribution)
- cross-entropy: fancy name for the log-likelihood of binary regression

ETC3250/5250 Lecture 6

Steps (2/2)

4. Training process:

- epochs: number of times the algorithm sees the entire data set
- batch_size: subset used at each iteration
- validation_split: proportion for hold-out set for computing error rate (a mini testing set)

5. Evaluation:

- usual metrics: accuracy, ROC, AUC (area under ROC curve), confusion table
- visualise the predictive probabilities
- examine misclassifications
- examine specific nodes, to understand what part it plays
- examine model boundary relative to the observed data

ETC3250/5250 Lecture 6

Example: penguins (1/5)

- 4D data
- Simple cluster structure to classes
- How many nodes in the hidden layer?

  Choose 2 nodes, because reducing to 2D, makes for easy classification.

  Counting the lines shows we have 19 parameters

The output layer and output function

While you can choose the number of hidden layers, nodes in each hidden layer and activation function (your NN architecture).

The output layer and output function are defined by the problem

The NN is applying a (possibly unbounded) non-linear function to unbounded data, so without an output function the NN would produce a value in $\mathbb{R}$

ETC3250/5250 Lecture 6

The output layer and output function

- Continuous regression, we want to predict a number in $\mathbb{R}$, so we can use the identify output function $\sigma(x) = x$
- If we wanted to predict several continuous things e.g. (size of the tip, size of the bill and how long they will be at the table) then we need an output node for each of these!
- What if your response variable was positive? Either exponential $\sigma(x) = \exp(x)$ or softplus function $\sigma(x) = \log(1 + \exp(x))$

ETC3250/5250 Lecture 6

The output layer and output function

- Binary classification we need to predict one number between 0 and 1 so we have one output node and use the logistic output function $\sigma(x) = \frac{1}{1+\exp(-x)}$
- Multiclass classification we need to predict K numbers between 0 and 1 and sum to 1. We have K output nodes and use the softmax output function $\sigma(x)_j = \frac{\exp(x_j)}{\sum_{k=1}^{K} \exp(x_k)}$

ETC3250/5250 Lecture 6

Example: penguins (2/5)

```r
library(keras3)
tensorflow::set_random_seed(211)
```

```r
# Define model
p_nn_model <- keras_model_sequential()
p_nn_model |>
  layer_dense(units = 2, activation = 'relu',
              input_shape = 4) |>
  layer_dense(units = 3, activation = 'softmax')
```

```r
p_nn_model |> summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 2)	10
dense_1 (Dense)	(None, 3)	9

```
Total params: 19 (76.00 B)
```

```
Trainable params: 19 (76.00 B)
Non-trainable params: 0 (0.00 B)
```

```r
loss_fn <- loss_sparse_categorical_crossentropy()
```

```r
p_nn_model |> compile(
  optimizer = "adam",
  loss      = loss_fn,
  metrics   = c('accuracy')
)
```

Split the data into training and test, and check it.

`tidymodels` style does not allow easy extraction of model coefficients - **https://parsnip.tidymodels.org/reference/mlp.html**

ETC3250/5250 Lecture 6

Example: penguins (3/5)

Fit the model

```
# Data needs to be matrix, and response needs to be numer
p_train_x <- p_train |>
  select(bl:bm) |>
  as.matrix()
p_train_y <- p_train |> pull(species) |> as.numeric()
p_train_y <- p_train_y-1 # Needs to be 0, 1, 2
p_test_x <- p_test |>
  select(bl:bm) |>
  as.matrix()
p_test_y <- p_test |> pull(species) |> as.numeric()
p_test_y <- p_test_y-1 # Needs to be 0, 1, 2
```

If you had categorical predictors you would need to turn these into dummies/one-hot-encode

```
p_train_x[1:3,]
```

```
         bl   bd    fl    bm
[1,] -0.88 0.78 -1.42 -0.56
```

```
[2,] -0.66 0.43 -0.42 -1.19
[3,] -0.85 1.75 -0.78 -0.69
```

```r
p_train_y[c(1:3, 100:103, 150:153)]
```

```
[1] 0 0 0 0 1 1 1 2 2 2 2
```

Notice the data has also been standardsied. This makes learning the model parameters easier and allows us to interpret their magnitudes

```r
# Fit model
p_nn_fit <- p_nn_model |>
  keras3::fit(
    x = p_train_x,
    y = p_train_y,
    #epochs = 500,
    epochs = 150,
    batch_size = 32, # default
    #verbose = 1
    verbose = 0
  )
```

Note if you run this twice, the second run starts from where the first finished

ETC3250/5250 Lecture 6

ETC3250/5250 Lecture 6

Diagnoing Convergence

While linear model parameter fitting is *exact* (you can turst R)
NN model fitting is **NOT**
The onus is on **YOU**, the data scientist/machine learner, to check your parameter fitting has *converged*
We use the in-sample error to do this (as this is what training was tryitng to make small)

`keras` plots the in-sample loss with the number of `epochs` for you
Check that this has become flat, and doesn't appear to still be going down
Then your training has *converged*

Other things to check

- If I get a different seed do I get a much better loss
- If I try a model with more nodes, do I get a smaller *in-sample* loss

ETC3250/5250 Lecture 6

Diagnoing Convergence

ETC3250/5250 Lecture 6

Diagnoing Convergence

ETC3250/5250 Lecture 6

Example: penguins (4/5)

Evaluate the fit

```
p_nn_model |>
  evaluate(p_test_x, p_test_y, verbose = 0)
```

```
$accuracy
[1] 0.8

$loss
[1] 0.63
```

Confusion matrices for training and test

```
           p_train_pred_cat
            Adelie Chinstrap Gentoo
  Adelie        96         3      1
  Chinstrap      2        43      0
  Gentoo        38         0     44


           p_test_pred_cat
            Adelie Chinstrap Gentoo
  Adelie        40         2      0
```

```
Chinstrap       0         23       0
Gentoo         20          1      20
```

Really good practice to look at both training and testing error here

Note: Specifically have chosen settings so fit is not perfect - This NN's training had not converged!

Estimated parameters

```r
# Extract hidden layer model weights
p_nn_wgts <- keras3::get_weights(p_nn_model, trainable=TR
p_nn_wgts
```

```
[[1]]
      [,1]  [,2]
[1,]  0.63  1.36
[2,]  0.17  0.08
[3,] -0.11 -0.30
[4,] -0.77 -0.24

[[2]]
[1]  0.118 -0.045

[[3]]
      [,1] [,2]  [,3]
[1,] -0.36  1.4 -1.73
[2,] -0.85  1.1  0.38
```



```
[[4]]
```

Which variables are contributing most to each hidden layer node?

Can you write out the model?

ETC3250/5250 Lecture 6

Example: penguins (5/5)

The values outputed by the hidden layer nodes are the *learned* engineered features.

This is the dimension reduction induced by the model.
Realistically, with a complex neural network, it is too much work to check these nodes.

Examine the predictive probabilities

The problem with this model is that Gentoo are too confused with Adelie. This is a structural problem because from the visualisation of the 4D data we know that there is a big gap between the Gentoo and both other species.

Deciding on a Architecture

Ideally, you would learn the model hyperparameters (#Hidden layer, #Nodes and activations) via cross-validation (as we did with trees)
This was feasible for trees as they were fast and easy to fit
NN's are slow and complicated to fit
I would advise hand tuning them by cross-validation i.e. do cross-validation but rather than trying a huge grid like we did with DTs, gradually make the model more complex.

- Stick to ReLU activations
- Start with a small number of nodes and train till convergence
  - keep track of in- and out-of-sample errors
- Increase the number of nodes
  - make sure the in-sample errors go down (if not you either need more training iterations or a new starting point)
  - Hopefully the out-of-sample error goes down too (if not you are overfitting)

- Consider adding another hidden layer and repeat

ETC3250/5250 Lecture 6

Deciding on a Architecture

Ideally, you would learn the model hyperparameters (#Hidden layer, #Nodes and activations) via cross-validation (as we did with trees)
This was feasible for trees as they were fast and easy to fit
NN's are slow and complicated to fit
I would advise hand tuning them by cross-validation i.e. do cross-validation but rather than trying a huge grid like we did with DTs, gradually make the model more complex.

Ideally the number of hidden nodes is greater than the numebr of predictors, the idea is to embed the predictors in a high dimensional space where they will be easier to classify.

But with many hidden nodes it is very easy to overfit!

Want to learn more?

- Take ETC3555/ETC5555 Statistical Machine Learning
  - We cover foundationation so Neural Network
  - Algorithms for learning their parameters
  - Ways to prevent overfitting
  - Extensions to image classification and text analysis
- Work your way through the example of fitting the **fashion MNIST data** using tensorflow.
- **Hands-on machine learning** has a lovely step-by-step guide to constructing and fitting.
- This is a very nice slide set: **A gentle introduction to deep learning in R using Keras**
- And the tutorials at **TensorFlow for R** have lots of examples.

ETC3250/5250 Lecture 6

Image Data

Neural Networks have been shown to have excellent performance in image classification e.g.

- Does this image contain a cat or a dog?
- Which number between 0 and 9 is this?

However, image data doesn't quite look like the normal predictor $x \in \mathbb{R}^p$ that we are now used to see

Well actually they do…

```
mnist<-dataset_mnist()
tri<-train_images<-mnist$train$x
train_labels<-mnist$train$y
tri[1,,]
```

```
     [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
[1,]    0    0    0    0    0    0    0    0    0     0
[2,]    0    0    0    0    0    0    0    0    0     0
[3,]    0    0    0    0    0    0    0    0    0     0
[4,]    0    0    0    0    0    0    0    0    0     0
[5,]    0    0    0    0    0    0    0    0    0     0
```

```
 [6,]    0    0    0    0    0    0    0    0    0    0
 [7,]    0    0    0    0    0    0    0    0   30   36
 [8,]    0    0    0    0    0    0    0   49  238  253
 [9,]    0    0    0    0    0    0    0   18  219  253
[10,]    0    0    0    0    0    0    0    0   80  156
[11,]    0    0    0    0    0    0    0    0    0   14
[12,]    0    0    0    0    0    0    0    0    0    0
[13,]    0    0    0    0    0    0    0    0    0    0
[14,]    0    0    0    0    0    0    0    0    0    0
```

ETC3250/5250 Lecture 6

Image Data

```
plot(as.raster(tri[1,,],max=255))
```

- Images are divided into pixels
- Greyscale images are $K_1 \times K_2$ dimensional matrices
- Each x_{jk} is the pixel density at pixels j, k with 0 being white and 255 being black
- This matrix is then 'flattened' into a $p = K_1 \times K_2$ dimensional vector
- On a computer, any colour can be represented as a mixture of Red, Green and Blue (RGB)
- Therefore each pixel now has 3 numbers, one for red, one for green and one for blue
- The image is now a $K_1 \times K_2 \times 3$ dimensional array

ETC3250/5250 Lecture 6

Next: Explainable artificial intelligence (XAI)

ETC3250/5250 Lecture 6